

Laboratorio 2 - Arreglos y excepciones

AGENCIA DE DETECTIVES – CASO MISTERIOSO

Objetivo

- Aplicar arreglos básicos para almacenar una cantidad fija de objetos dentro de un programa.
- Acceder y modificar elementos de un arreglo mediante índices.
- Recorrer arreglos de objetos utilizando estructuras repetitivas y validar correctamente posiciones que contengan null.
- Utilizar ArrayList para almacenar y administrar una cantidad dinámica de objetos.
- Realizar operaciones de registro, búsqueda, modificación, eliminación y cálculo utilizando los datos almacenados.
- Implementar manejo de excepciones utilizando try-catch y finally.
- Utilizar excepciones para controlar entradas incorrectas y valores que no cumplan con las reglas establecidas.
- Aplicar de forma correcta los modificadores de visibilidad para no violar el encapsulamiento.
- Modelar el programa utilizando UML.
- Crear un programa principal que resuelva el problema utilizando programación orientada a objetos.

Introducción

Una agencia privada de detectives ha recibido un nuevo caso y necesita desarrollar un sistema que permita organizar la información recopilada durante la investigación.

Al iniciar el caso, los investigadores han identificado una cantidad fija de posibles ubicaciones que deberán investigarse. Algunas de estas ubicaciones todavía no han sido descubiertas o registradas, por lo que los espacios correspondientes podrán encontrarse vacíos. Debido a que la cantidad máxima de ubicaciones está definida desde el inicio, deberán administrarse utilizando un arreglo básico.

Durante la investigación pueden encontrarse una cantidad desconocida de pistas. Algunas pistas podrán agregarse, modificarse o descartarse conforme avance el caso. Por esta razón, las pistas deberán almacenarse utilizando un ArrayList.

El sistema deberá permitir administrar ambas estructuras y manejar correctamente situaciones que puedan producir errores durante la ejecución del programa.

Para administrar la investigación se debe realizar lo siguiente:

1. Al iniciar el programa se debe solicitar la información del caso. Como mínimo se debe ingresar:
 - a. Nombre del caso.
 - b. Código de identificación.
 - c. Nombre del detective responsable.
2. Cada caso contará con un máximo de **5 ubicaciones de investigación**. Las ubicaciones registradas deberán almacenarse utilizando un **arreglo básico de objetos**.
3. Cada ubicación deberá almacenar como mínimo:
 - a. Código.

- b. Nombre.
 - c. Dirección o descripción del lugar.
 - d. Nivel de riesgo.
 - e. Estado.
4. El nivel de riesgo deberá encontrarse entre 1 y 10. Si se intenta crear o modificar una ubicación utilizando un nivel menor que 1 o mayor que 10, deberá generarse una `IllegalArgumentException`.
 5. Al registrar una nueva ubicación, el usuario deberá indicar la posición del arreglo donde desea almacenarla. Antes de realizar la operación se deberá validar que:
 - a. La posición indicada se encuentre dentro de los límites del arreglo.
 - b. La posición seleccionada se encuentre disponible.
 - c. La ubicación posea información válida.
 6. Las posiciones del arreglo que todavía no tengan una ubicación registrada contendrán null. El programa deberá verificar esta condición antes de intentar utilizar los métodos de una ubicación.
 7. El sistema deberá permitir consultar todas las ubicaciones registradas. No deberán mostrarse como ubicaciones existentes aquellas posiciones del arreglo que contengan null.
 8. El sistema deberá permitir consultar una ubicación específica indicando la posición que ocupa dentro del arreglo. Si la posición no es válida o se encuentra vacía, deberá informarse al usuario sin finalizar inesperadamente el programa.
 9. El sistema deberá permitir modificar el nivel de riesgo y el estado de una ubicación previamente registrada. No se podrá modificar una posición que contenga null y el nuevo nivel de riesgo deberá encontrarse entre 1 y 10.
 10. El sistema deberá permitir descartar una ubicación previamente registrada. Al descartarla, la posición correspondiente deberá volver a contener null.
 11. Durante la investigación podrá encontrarse una cantidad desconocida de pistas. Las pistas encontradas deberán almacenarse utilizando obligatoriamente un `ArrayList`.
 12. Cada pista deberá almacenar como mínimo:
 - a. Código.
 - b. Descripción.
 - c. Tipo de evidencia.
 - d. Nivel de importancia.
 - e. Nivel de confiabilidad.
 13. El nivel de importancia deberá encontrarse entre 1 y 10 y el nivel de confiabilidad deberá encontrarse entre 0 y 100. Si alguno de estos valores no cumple con las reglas establecidas, deberá generarse una `IllegalArgumentException`.
 14. El sistema deberá permitir registrar nuevas pistas utilizando el `ArrayList`. No se podrán registrar dos pistas con el mismo código.
 15. El sistema deberá permitir consultar todas las pistas encontradas. Si todavía no existen pistas registradas, deberá manejarse correctamente la situación.
 16. El sistema deberá permitir buscar una pista mediante su código. Para realizar la búsqueda deberá recorrerse el `ArrayList`.
 17. El sistema deberá permitir modificar la información de una pista previamente registrada. Los nuevos valores deberán cumplir las mismas validaciones utilizadas durante su creación.
 18. El sistema deberá permitir eliminar una pista que haya sido descartada o determinada como falsa. Para ello deberá localizarse la pista correspondiente y eliminarla del `ArrayList`.

19. El sistema deberá calcular y mostrar:
 - a. Cantidad de ubicaciones registradas.
 - b. Cantidad de espacios disponibles para ubicaciones.
 - c. Ubicación con mayor nivel de riesgo.
 - d. Cantidad de pistas registradas.
 - e. Pista con mayor nivel de importancia.
 - f. Pista con mayor nivel de confiabilidad.
 - g. Promedio del nivel de importancia de las pistas registradas.
20. Para los cálculos relacionados con pistas únicamente deberán considerarse los objetos registrados dentro del ArrayList. Si este se encuentra vacío, el programa deberá evitar realizar cálculos que requieran elementos existentes.
21. Cuando el programa solicite un valor numérico mediante Scanner y el usuario ingrese un dato que no corresponda al tipo solicitado, deberá manejarse la InputMismatchException. El programa deberá limpiar la entrada incorrecta y permitir que el usuario continúe utilizando el sistema.
22. El programa deberá utilizar bloques try-catch para manejar las excepciones que puedan producirse durante la interacción con el usuario y las validaciones de los objetos.
23. El programa deberá utilizar al menos un bloque finally dentro de una operación donde tenga sentido ejecutar una acción final independientemente de que la operación se complete correctamente o se produzca una excepción.
24. El programa deberá continuar funcionando después de una entrada incorrecta o una excepción que haya sido manejada.

Ejercicio

Análisis

Responda las siguientes preguntas:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?
4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?
5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
6. ¿Qué parámetros serán requeridos por los métodos en sus clases?
7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?
8. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?
9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?
10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Diseño

Desarrolle un diagrama de clases que muestre los miembros de cada clase y que establezca las relaciones necesarias entre ellas.

Incluya en su diagrama al driver program, que contiene el método main().

El diagrama deberá representar correctamente los atributos, métodos, tipos de datos, parámetros y modificadores de visibilidad identificados durante el análisis.

El diagrama deberá representar correctamente las propiedades que utilizan el arreglo básico y el ArrayList.

Use anotaciones (recuadros de texto) en su diagrama para aclarar cualquier información que considere necesaria.

Programa

Desarrolle un sistema que contenga un driver program y las instancias necesarias de las clases identificadas durante su análisis.

El programa deberá utilizar como mínimo un **arreglo básico de objetos** para almacenar las ubicaciones de investigación y un **ArrayList de objetos** para almacenar las pistas encontradas.

El programa deberá implementar manejo de excepciones mediante try-catch y finally, de acuerdo con las situaciones identificadas durante el análisis.

Así mismo, debe mostrar un menú con las siguientes opciones:

1. **Nuevo caso:** reemplaza la instancia actual del caso. Al crear un nuevo caso, este comienza sin ubicaciones registradas y sin pistas encontradas.
2. **Registrar ubicación:** solicita la posición donde se desea registrar la ubicación y la información necesaria para crearla. La posición debe estar disponible.
3. **Consultar ubicaciones:** muestra todas las ubicaciones registradas actualmente y las posiciones que ocupan dentro del arreglo.
4. **Consultar una ubicación:** solicita una posición y muestra la información de la ubicación almacenada en ella. Si la posición está vacía, deberá indicarse al usuario.
5. **Modificar ubicación:** solicita la posición de una ubicación previamente registrada y permite modificar su nivel de riesgo y estado.
6. **Descartar ubicación:** solicita la posición de una ubicación y la elimina del arreglo, dejando nuevamente disponible dicha posición.
7. **Registrar pista:** solicita la información de una nueva pista y la agrega al ArrayList. No deberá permitirse registrar códigos repetidos.
8. **Consultar pistas:** muestra todas las pistas encontradas durante la investigación.
9. **Buscar pista:** solicita el código de una pista y muestra su información si se encuentra registrada.
10. **Modificar pista:** solicita el código de una pista previamente registrada y permite modificar su información.
11. **Eliminar pista:** solicita el código de una pista y la elimina del ArrayList.
12. **Mostrar reporte de investigación:** muestra la cantidad de ubicaciones registradas y

espacios disponibles, la ubicación con mayor nivel de riesgo, la cantidad de pistas encontradas, la pista con mayor nivel de importancia, la pista con mayor nivel de confiabilidad y el promedio del nivel de importancia.

13. **Salir:** termina la ejecución del programa.

El programa deberá validar las entradas necesarias para evitar almacenar información inválida, acceder a posiciones inexistentes del arreglo, utilizar referencias null o realizar operaciones inválidas sobre un ArrayList vacío.

Las entradas numéricas incorrectas deberán manejarse mediante InputMismatchException. Los valores que no cumplan con las reglas establecidas para los objetos deberán manejarse mediante IllegalArgumentException.

El programa deberá utilizar ArrayList directamente para la colección dinámica. **No deberá utilizarse List.**

Material a entregar en Canvas

- Archivo PDF con el análisis y diseño.
 - No incluya únicamente la URL del diagrama; adjunte el diagrama dentro del PDF o como un archivo externo.
 - Use la plantilla proporcionada.
- URL al repositorio público en GitHub conteniendo todos los archivos .java necesarios para ejecutar el programa.

Evaluación

- **Análisis (30 puntos)**
 - **(5 puntos)** Se listan correctamente todos los requisitos funcionales del sistema a desarrollar, punto por punto.
 - **(5 puntos)** Se describe el propósito de cada una de las clases identificadas. La descripción es lógica y coherente con lo que hace la clase dentro del sistema que modela y resuelve el problema presentado.
 - **(10 puntos)** Se describe el propósito de cada atributo de cada clase. La descripción es suficiente para comprender la razón de la creación del atributo y su importancia como propiedad de las instancias o clases. Se identifican correctamente las propiedades que utilizan el arreglo básico y el ArrayList.
 - **(10 puntos)** Se describe el propósito de la creación de cada método de cada clase. La descripción es suficiente para comprender qué acciones realizará cada método, qué información requerirá para funcionar y qué resultados producirá. Se identifican las operaciones que pueden producir excepciones.
- **Diseño (30 puntos)**
 - **(8 puntos)** La representación de las clases coincide con el estándar del lenguaje UML.
 - **(12 puntos)** Están representados cada uno de los componentes de las clases de manera correcta y coinciden con lo descrito en el análisis.
 - **(10 puntos)** Las relaciones entre las clases están correctamente representadas.
- **Programa (40 puntos)**
 - **(10 puntos)** La cantidad de clases identificadas es suficiente para resolver el problema planteado y se utilizan correctamente los principios básicos de programación orientada a objetos.

- **(10 puntos)** Se implementa correctamente un arreglo básico de objetos para almacenar las ubicaciones y se realizan las operaciones solicitadas utilizando índices, estructuras repetitivas y validación de posiciones null.
- **(10 puntos)** Se implementa correctamente un ArrayList para almacenar las pistas y se realizan las operaciones solicitadas de registro, búsqueda, modificación, eliminación y cálculo.
- **(10 puntos)** Se implementa correctamente el manejo de excepciones utilizando try-catch y finally. Se manejan las entradas incorrectas y los valores inválidos sin provocar la finalización inesperada del programa.